

TECHNOLOGIES IN EDUCATION
UNIVERSITY NSU

MICROELECTRONICS
INNOVATIONS
CATALYTIC
MATERIALS
ASSEMBLY
POINT

SCIENTIFIC
LABORATORY
HYBRID
MATERIALS
GEOPHYSICS
ENGINEERING
ENERGY CONSERVATION
BIOTECHNOLOGY
GEOCHEMISTRY
NANOTECHNOLOGY

HIGH
ENERGIES
SEMIOTICS
SCIENCE
MATHEMATICAL MODELING

DEVELOPMENT
ELEMENTARY
PARTICLES

THE ARCTIC REGIONS
DARK
MATTER

QUANTUM
TECHNOLOGIES
BIOMEDICINE

APPLIED
STUDIES
PHOTONICS

ASTRONOMY
GLOBAL PRIORITY

ASTROPHYSICS
BIOINFORMATICS

LASER
PHYSICS

KNOWLEDGE
ECONOMY

GEOLOGY
ARCHEOLOGY

IT
DEEP
LEARNING
BRAIN
STUDY

COGNITIVE TECHNOLOGIES

N* Novosibirsk
State
University
*THE REAL SCIENCE

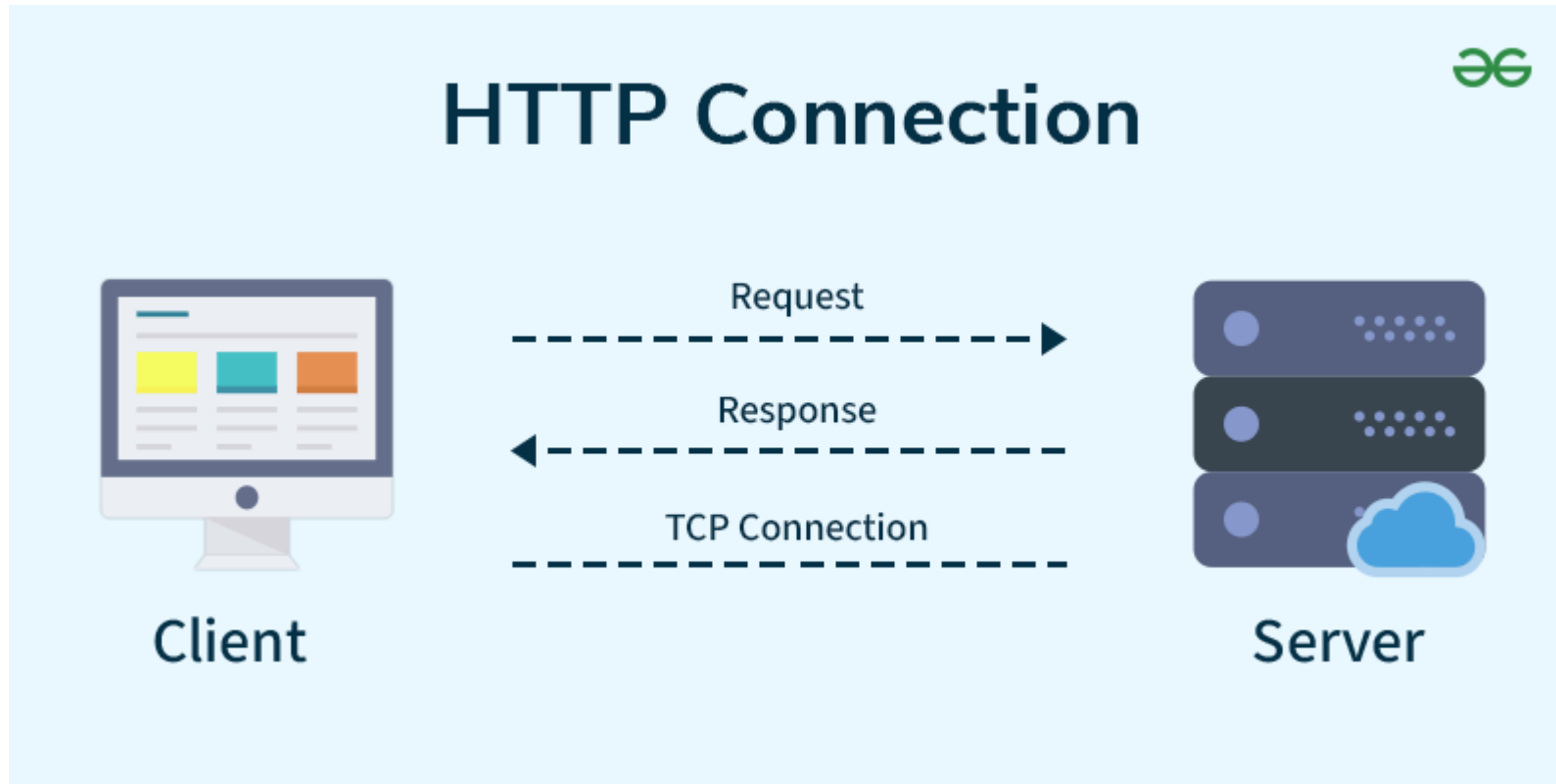
Машинное обучение

Семинар 13

Глушенко Андрей Валерьевич
Институт Интеллектуальной
Робототехники

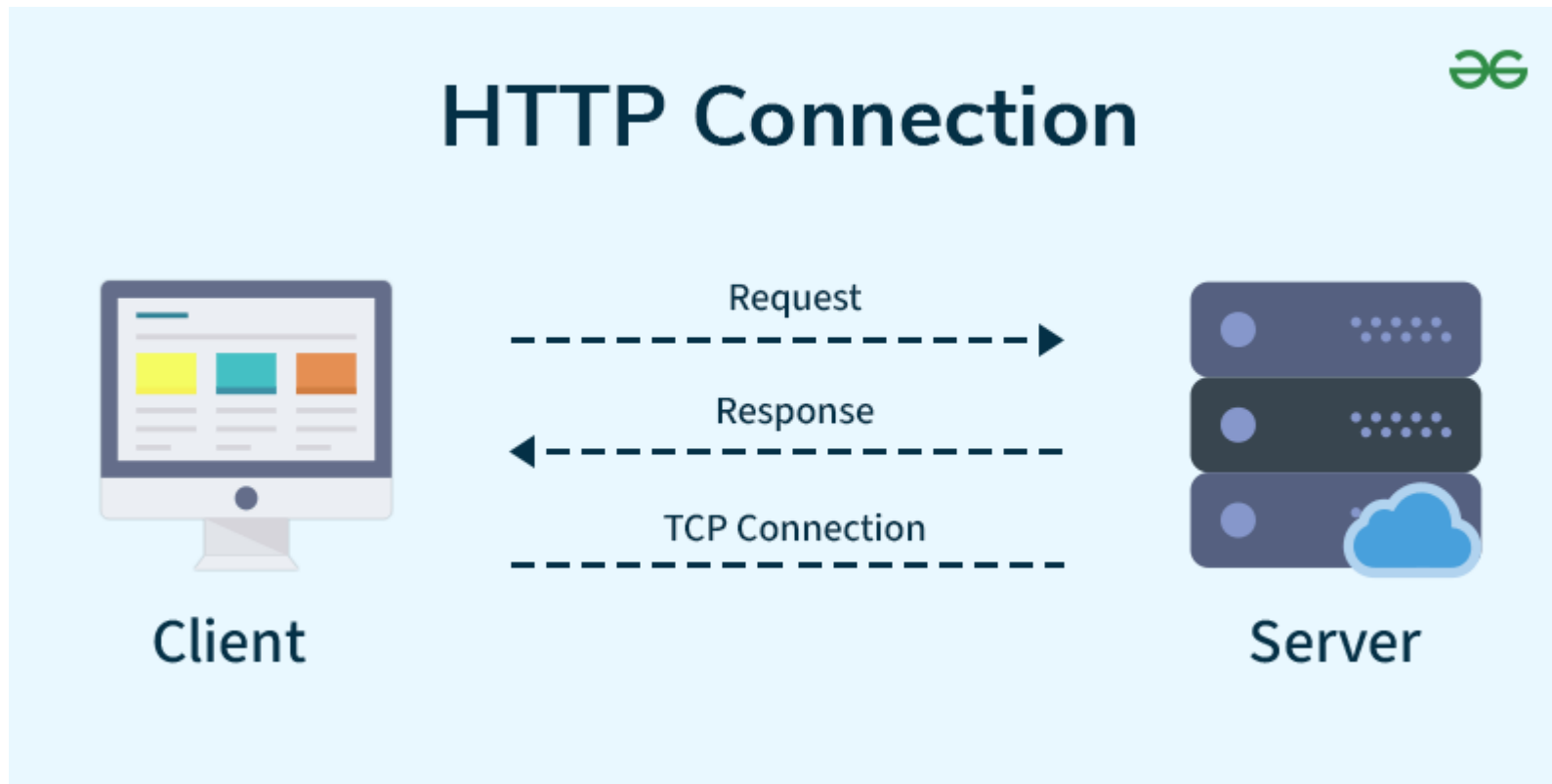
HTTP (HyperText Transfer Protocol)

HTTP — это прикладной протокол передачи данных, изначально предназначенный для передачи гипертекста, но ставший универсальным стандартом для обмена данными в распределенных информационных системах.



HTTP (HyperText Transfer Protocol)

- **Модель взаимодействия:** Клиент-серверная архитектура на базе парадигмы «запрос-ответ» (request-response).
- **Транспортный уровень:** Обычно использует стек TCP/IP (порт 80 для HTTP и 8443 для HTTPS).



HTTP (HyperText Transfer Protocol)

Структура сообщения:

- **Стартовая строка (Start-line):** Метод (GET, POST и т.д.), URI ресурса и версия протокола.
- **Заголовки (Headers):** Метаданные (тип контента, кодировка, параметры авторизации, куки).
- **Пустая строка:** Разделитель между метаданными и телом.
- **Тело сообщения (Body):** Непосредственно данные (JSON, XML, HTML, бинарные файлы).

HTTP (HyperText Transfer Protocol)

Коды состояния (Status Codes): Сервер возвращает трехзначный код, определяющий результат обработки:

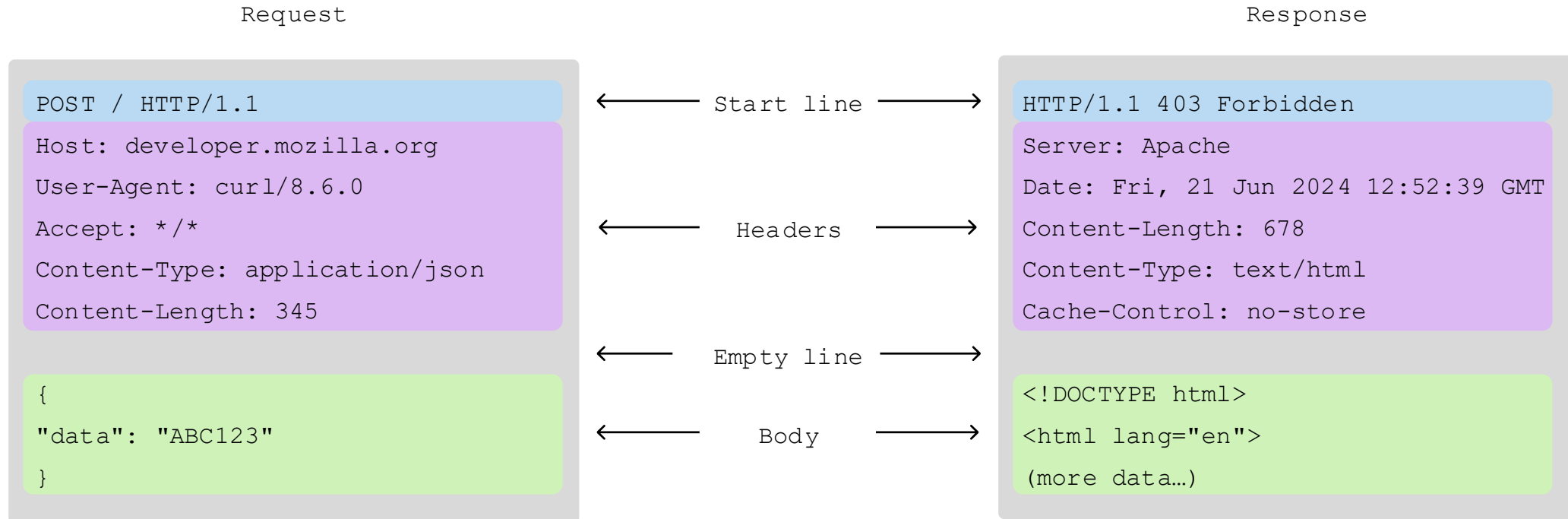
2xx – Успех (например, 200 OK, 201 Created).

3xx – Перенаправление.

4xx – Ошибка клиента (например, 404 Not Found, 401 Unauthorized).

5xx – Ошибка сервера (например, 500 Internal Server Error).

HTTP (HyperText Transfer Protocol)



REST (Representational State Transfer)

REST – это архитектурный стиль проектирования распределенных систем, сформулированный Роем Филдингом в 2000 году. Это не протокол и не стандарт, а совокупность ограничений (constraint), накладываемых на систему.

6 ключевых ограничений REST:

1. **Client-Server (Клиент-серверная модель):** Разделение интерфейса пользователя и логики хранения данных. Это повышает переносимость кода интерфейса и масштабируемость серверной части.
2. **Stateless (Отсутствие состояния):** Состояние сессии хранится полностью на стороне клиента. Сервер не должен сохранять контекст между запросами; каждый запрос самодостаточен и содержит всю информацию для его идентификации и выполнения.
3. **Cacheable (Кэширование):** Данные ответа должны иметь явную пометку о возможности их кэширования, чтобы снизить нагрузку на сеть и сервер.

6 ключевых ограничений REST:

4. Uniform Interface (Единообразие интерфейса):

- **Идентификация ресурсов:** Каждый ресурс имеет уникальный URI.
- **Манипуляция ресурсами через представления:** Клиент взаимодействует с ресурсом через его представление (например, JSON-документ).
- **Самодокументируемые сообщения:** Сообщение содержит достаточно информации, чтобы понять, как его обработать (через MIME-типы).
- **HATEOAS:** Клиент получает информацию о доступных действиях через гиперссылки, возвращаемые сервером.

6 ключевых ограничений REST:

5. Layered System (Многоуровневая система): Клиент не знает, подключен ли он напрямую к серверу или к промежуточному узлу (балансировщику, прокси), что позволяет наращивать сложность системы без изменения клиента.

6. Code on Demand (Код по запросу — необязательно): Возможность передачи исполняемого кода (например, JavaScript-скриптов) от сервера клиенту.

REST

В REST всё является ресурсом. Ресурс — это абстрактная сущность, имеющая состояние. Состояние ресурса передается в различных форматах (Representation), наиболее популярным из которых сегодня является JSON. Взаимодействие с ресурсами осуществляется строго через стандартные методы протокола HTTP, что делает API предсказуемым.

React

- React — это декларативная JavaScript-библиотека с открытым исходным кодом для разработки пользовательских интерфейсов, основанная на компонентном подходе. С архитектурной точки зрения React фокусируется на уровне View (представление) в паттерне MVC (хотя в современной экосистеме он чаще рассматривается как база для построения сложных SPA — Single Page Applications).

Компонентная архитектура (Component-Based)

- Основной структурной единицей React является компонент. Это независимый, инкапсулированный модуль кода, который описывает часть интерфейса и обладает собственной логикой.
- Иерархичность: Приложение строится как дерево компонентов (Root -> Parent -> Child).
- Изоляция: Компоненты минимизируют побочные эффекты, взаимодействуя друг с другом через строго определенные интерфейсы (props).

Декларативный подход

- В отличие от императивного программирования (где описывается как изменить DOM шаг за шагом), React использует декларативный стиль. Разработчик описывает конечное состояние интерфейса для текущих данных. При изменении данных React самостоятельно берет на себя задачу по эффективному обновлению DOM.

Virtual DOM (Виртуальная объектная модель документа)

- Это ключевой механизм оптимизации производительности.
- Суть: React создает в памяти легковесную копию реального DOM.
- Процесс дифференциации (Reconciliation): При изменении состояния (state) создается новое дерево Virtual DOM. React сравнивает его с предыдущей версией (алгоритм Diffing), вычисляет минимально необходимый набор изменений и применяет их к реальному браузерному DOM одним пакетом. Это снижает количество дорогостоящих операций перерисовки.

Однонаправленный поток данных (One-Way Data Flow)

- Данные в React-приложении передаются строго сверху вниз по дереву компонентов.
- Props (Свойства): Неизменяемые (read-only) входные данные, передаваемые от родителя к потомку.
- State (Состояние): Внутренние динамические данные компонента, изменение которых инициирует повторный рендеринг.
- Такой подход делает поток данных предсказуемым и упрощает отладку.

JSX (JavaScript XML)

- Синтаксическое расширение языка JavaScript, которое позволяет описывать структуру интерфейса с использованием XML-подобного синтаксиса непосредственно внутри JS-кода. Транспилаторы (например, Babel) преобразуют JSX в вызовы функций `React.createElement()`.

Хуки (Hooks)

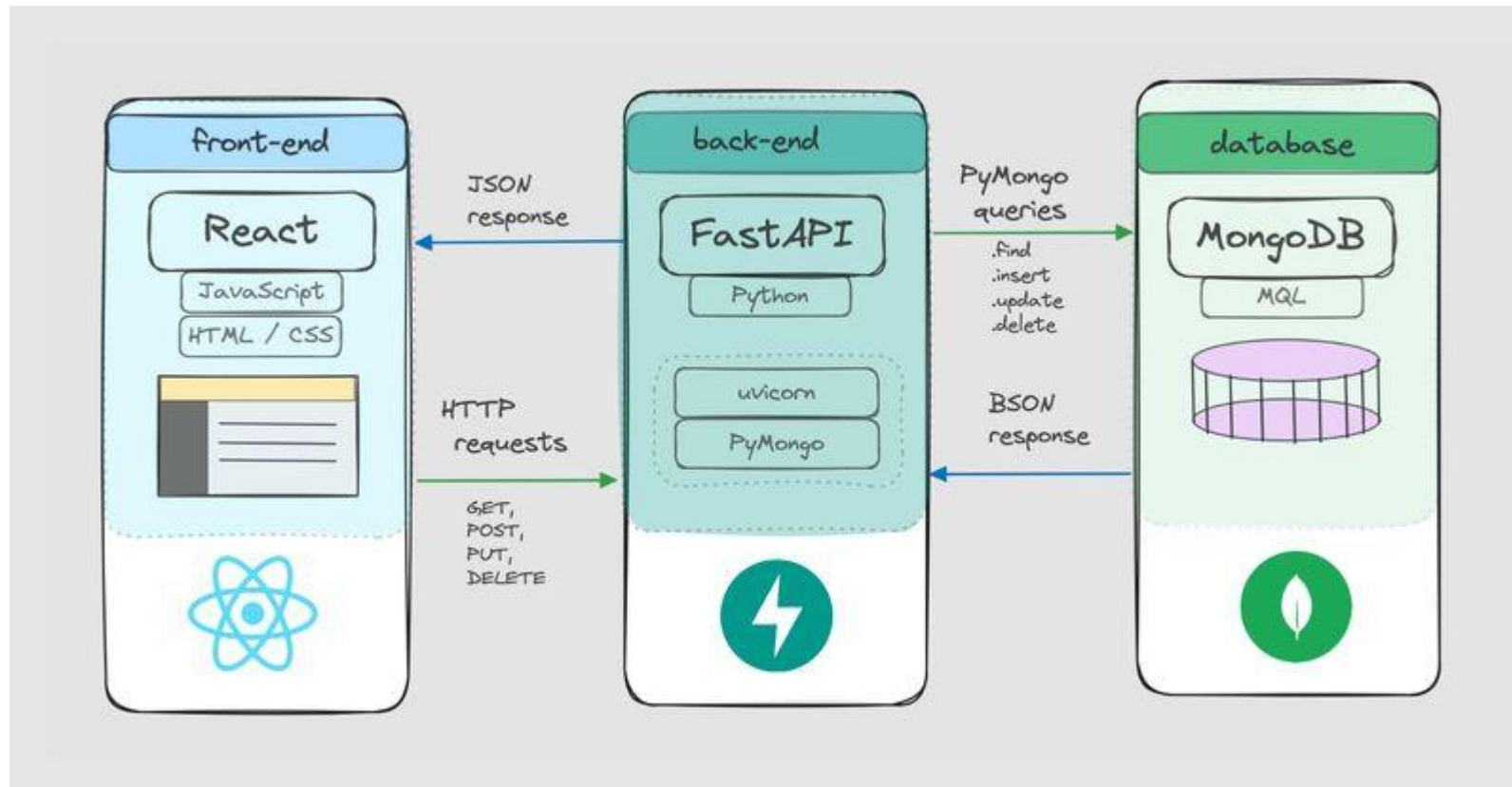
- Введены в версии 16.8 как механизм для использования состояния и других возможностей React в функциональных компонентах (вытеснив классовый подход).
- **useState**: управление локальным состоянием.
- **useEffect**: управление побочными эффектами (запросы к API, подписки, манипуляции с DOM вне цикла React).
- **useContext**: доступ к глобальным данным (Context API) без «прокидывания» пропсов через все уровни иерархии.

Экосистема и жизненный цикл

- **Управление состоянием:** Redux, MobX, Zustand или встроенный Context API.
- **Маршрутизация:** React Router (для реализации клиентского роутинга в SPA).
- **Серверный рендеринг (SSR):** Фреймворки вроде Next.js для генерации HTML на стороне сервера, что критично для SEO и производительности.

Взаимодействие с бэкендом

- Взаимодействие React (фронтенд) и FastAPI (бэкенд) строится на базе протокола **HTTP** и архитектурного стиля **REST**. Процесс обмена данными формально можно разделить на несколько этапов.



Инициирование запроса (Client Side)

- React-приложение, работающее в браузере, отправляет асинхронный HTTP-запрос к серверу. Для этого используются:
- **Fetch API**: встроенный в браузер стандартный метод.
- **Axios**: популярная библиотека, предоставляющая более удобный интерфейс (автоматическое преобразование в JSON, перехватчики запросов и т.д.).
- Запрос обычно иницируется внутри хука `useEffect` (при загрузке компонента) или в обработчике события (например, клик по кнопке).

Маршрутизация и обработка (Server Side - FastAPI)

- FastAPI ожидает запрос на определенном Endpoint (конечной точке).
- **Декораторы:** В коде FastAPI вы определяете метод и путь: `@app.get("/items/{item_id}")`.
- **Pydantic:** FastAPI использует библиотеку Pydantic для автоматической валидации входящих данных. Если фронтенд отправит строку вместо ожидаемого числа, бэкенд автоматически вернет ошибку `422 Unprocessable Entity`.

Маршрутизация и обработка (Server Side - FastAPI)

- FastAPI ожидает запрос на определенном Endpoint (конечной точке).
- **Декораторы:** В коде FastAPI вы определяете метод и путь: `@app.get("/items/{item_id}")`.
- **Pydantic:** FastAPI использует библиотеку Pydantic для автоматической валидации входящих данных. Если фронтенд отправит строку вместо ожидаемого числа, бэкенд автоматически вернет ошибку `422 Unprocessable Entity`.

Формат обмена данными

- Почти всегда используется JSON (JavaScript Object Notation).
- **Request:** React сериализует объект JavaScript в строку JSON и отправляет в теле запроса (для POST/PUT).
- **Response:** FastAPI сериализует объект Python (или модель базы данных) в JSON и отправляет обратно с соответствующим статус-кодом.

Политика CORS (Cross-Origin Resource Sharing)

- Это критически важный аспект при связке React и FastAPI.
- **Проблема:** По умолчанию браузеры блокируют запросы к домену, отличному от того, на котором запущен фронтенд (например, React на localhost:3000, а FastAPI на localhost:8000).
- **Решение:** В FastAPI необходимо настроить CORSMiddleware, чтобы разрешить фронтенду обращаться к API.

Политика CORS (Cross-Origin Resource Sharing)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()
# Настройка CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"], # Адрес фронтенда
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/api/data")
async def get_data():
    return {"message": "Hello from FastAPI", "status": "success"}
```

Александр Жадан

Александр Жадан — российский программист и эксперт в области искусственного интеллекта, получивший широкую известность благодаря нестандартному использованию чат-ботов на базе GPT для решения личных и академических задач



Александр Жадан

Он разработал систему на базе нейросетей, чтобы автоматизировать поиск партнерши в Tinder. Вот основные детали его эксперимента:

Масштаб отбора: Программа проанализировала анкеты и пообщалась с 5239 девушками.

Фильтрация: ИИ отсеивал профили по жестким критериям (например, отсутствие работы, наличие определенных знаков зодиака в описании или фото с цветами).

Александр Жадан

Автоматическое общение: ChatGPT вела переписку за Александра, имитируя его стиль. На ранних этапах случались курьезы: например, бот мог назначить свидание в лесу или пообещать прийти с шоколадкой, о чем сам Александр узнавал уже на месте.

Результат: Система отобрала несколько наиболее подходящих кандидаток, с которыми Александр начал встречаться лично. В итоге он выбрал девушку по имени Карина.

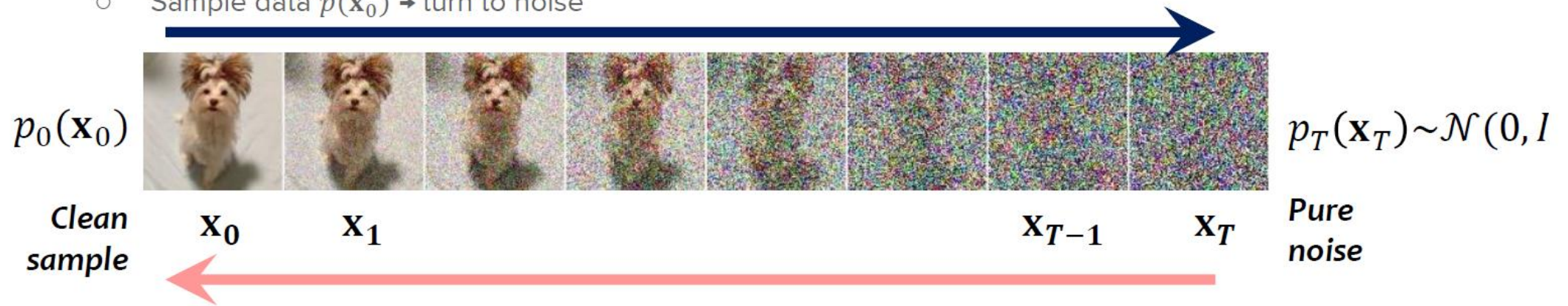
Предложение: Даже решение о свадьбе было принято с подачи ИИ. Нейросеть проанализировала их длительное общение и порекомендовала сделать предложение, так как отношения выглядели стабильными.

Диффузионная модель

Denoising diffusion models

- **Forward / noising process**

- Sample data $p(\mathbf{x}_0) \rightarrow$ turn to noise



- **Reverse / denoising process**

- Sample noise $p_T(\mathbf{x}_T) \rightarrow$ turn into data